

Machine Learning Systems

借助期中系统复习一下机器学习系统的相关内容。

参考 窦德景 老师的 slides.

2. 线性和逻辑回归，贝叶斯模型

2.1 Linear Regression

基础假设：

1. $\mathbb{E}(Y|X = x)$ can be expressed as a weighted sum of the features.
2. Noise follows a Gaussian distribution.
3. Features are independent.

常用表示： $x^{(i)} = [x_1^{(i)}, x_2^{(i)}, \dots, x_d^{(i)}]^\top$ and its corresponding label is $y^{(i)}$.

模型： $\hat{y} = Xw + b$

损失函数：

$$l^{(i)}(w, b) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2, L(w, b) = \frac{1}{n} \sum_{i=1}^n l^{(i)}(w, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2}(w^\top x^{(i)} + b - y^{(i)})^2$$

目标： $w^*, b^* = \underset{w, b}{\operatorname{argmin}} L(w, b)$

梯度：

$$\begin{aligned} \frac{\partial L(w, b)}{\partial w} &= \frac{1}{n} \sum_{i=1}^n \frac{\partial}{\partial w} l^{(i)}(w, b) = \frac{1}{n} \sum_{i=1}^n (w^\top x^{(i)} + b - y^{(i)}) x^{(i)} = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) x^{(i)} \\ \frac{\partial L(w, b)}{\partial b} &= \frac{1}{n} \sum_{i=1}^n \frac{\partial}{\partial b} l^{(i)}(w, b) = \frac{1}{n} \sum_{i=1}^n (w^\top x^{(i)} + b - y^{(i)}) = \frac{1}{n} \sum_{i=1}^n \hat{y}^{(i)} - y^{(i)} \end{aligned}$$

优化算法：小批量随机梯度下降(Minibatch Stochastic Gradient Descent)

$$(w, b) \leftarrow (w, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{(w, b)} l^{(i)}(w, b)$$

2.2 Logistic Regression

Logistic Regression 中虽然含有“回归”二字，但确是**分类方法**，不过它的本质的确是回归。下面是 logistic regression 的核心内容：

$$h_{\theta}(x) = g(\theta^{\top} x)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^{\top} x}} = p(y = 1|x; \theta)$$

$g(z)$ is also called sigmoid function.

损失函数：不采用平方损失函数，因为会导致难以解决的非凸优化问题。采用极大似然估计(MLE).

$$l(\theta) = \prod_{i=1}^n p(y^{(i)}|x^{(i)}; \theta) = \prod_{i=1}^n l^{(i)}(\theta) \Rightarrow \log l(\theta) = \sum_{i=1}^n \log l^{(i)}(\theta)$$

其中

$$\begin{aligned} \log l^{(i)}(\theta) &= y^{(i)} \log p(y^{(i)} = 1|x^{(i)}; \theta) + (1 - y^{(i)}) \log p(y^{(i)} = 0|x^{(i)}; \theta) \\ &= y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \end{aligned}$$

令 $J(\theta) = -l(\theta)$ ，故有损失函数（其实是交叉熵损失，专门用于刻画分类任务的损失）：

$$J(\theta) = - \sum_{i=1}^n [y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

$$\text{目标: } \min_{\theta} J(\theta) \rightarrow \min_{\theta} J_{\text{regularized}}(\theta) = J(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

由于

$$\frac{\partial h_{\theta}(x)}{\partial \theta} = h_{\theta}(x)[1 - h_{\theta}(x)]x$$

故

$$-\frac{\partial l(\theta)}{\partial \theta} = [h_{\theta}(x) - y]x$$

因此梯度为：

$$\frac{\partial J_{\text{regularized}}(\theta)}{\partial \theta} = \sum_{i=1}^n [h_{\theta}(x^{(i)}) - y^{(i)}]x^{(i)} + \lambda \theta$$

Softmax Regression:

$$p(y = c|x; \theta_1, \dots, \theta_C) = \frac{\exp(\theta_c^{\top} x)}{\sum_{i=1}^C \exp(\theta_i^{\top} x)}$$

在 logistic regression 中，相当于将 $y = 0$ 这一类对应的参数设置为 **0** 向量。

2.3 Naive Bayes

是基于 Bayes' Theorem 的**分类方法**，做出了 **条件独立性** 的假设，对于特别大的数据集上特别有用。核心内容：

$$p(c|x) = \frac{P(x|c)P(c)}{P(x)} \stackrel{\text{Conditional Independence}}{=} \frac{\prod_{i=1}^n P(x_i|c)P(c)}{P(x)}$$

在实际应用中，往往不需要计算分母，而是只计算分子，最后进行归一化。

3. SVM

假设线性可分。使用 $\text{sign}(w^\top x + b)$ 进行预测。

3.1 几何间隔与函数间隔

几何间隔： $\gamma_i = y_i \left(\frac{w^\top x_i + b}{\|w\|} \right)$ 从距离上定义，为正表示正例，为负表示负例。可以刻画划分的正确程度。

函数间隔： $\hat{\gamma}_i = y_i(w^\top x_i + b)$ 去掉了分母，便于处理。由于 $\|w\|$ 可以任意改变，因此仅符号有意义。

3.2 间隔最大化

优化目标：

$$\begin{aligned} & \max_{w,b} \gamma \\ \text{s.t. } & y_i \left(\frac{w^\top x_i + b}{\|w\|} \right) \geq \gamma, \quad i = 1, \dots, n \end{aligned}$$

等价于

$$\begin{aligned} & \max_{w,b} \frac{\hat{\gamma}}{\|w\|} \\ \text{s.t. } & y_i(w^\top x_i + b) \geq \hat{\gamma}, \quad i = 1, \dots, n \end{aligned}$$

由于函数间隔可以任取，仅符号有意义，故取 $\hat{\gamma} = 1$ ，有：

$$\begin{aligned} & \max_{w,b} \frac{1}{\|w\|} \\ \text{s.t. } & y_i(w^\top x_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

等价于

$$\begin{aligned} & \min_{w,b} \frac{1}{2} \|w\|^2 \\ \text{s.t. } & y_i(w^\top x_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

3.3 拉格朗日对偶

设 $\alpha_i \geq 0$ ，令拉格朗日函数为

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \alpha_i [1 - y_i(w^\top x_i + b)]$$

容易证明原优化问题等价于

$$\min_{w, b} \max_{\alpha} L(w, b, \alpha)$$

其对偶问题是凸优化问题：

$$\max_{\alpha} \min_{w, b} L(w, b, \alpha) = \max_{\alpha} \theta_p(\alpha), \quad \theta_p(\alpha) = \min_{w, b} L(w, b, \alpha)$$

但是容易证明对偶问题的解小于等于原问题的解，KKT 取等条件如下：

$$\left\{ \begin{array}{ll} \nabla_w L(w^*, b^*, \alpha^*) = w^* - \sum_i \alpha_i^* y_i x_i = 0 & \text{Stationarity Condition, 站点条件} \\ \nabla_b L(w^*, b^*, \alpha^*) = -\sum_i \alpha_i^* y_i = 0 & \\ \alpha_i^* [1 - y_i(w^{*\top} x_i + b^*)] = 0 & \text{Complementary Slackness Condition, 互补松弛条件} \\ y_i(w^{*\top} x_i + b^*) \geq 1 & \text{Primal Feasibility Condition, 原始可行性条件} \\ \alpha_i^* \geq 0 & \text{Dual Feasibility Condition, 对偶可行性条件} \end{array} \right.$$

其中站点条件说明 $w^* = \sum_i \alpha_i^* y_i x_i$ ，仅有 $\alpha_i > 0$ 在起作用。而由互补松弛条件可知，若 $\alpha_i^* > 0$ ，则有 $y_i(w^{*\top} x_i + b^*) - 1 = 0$ ，意味着这些起作用的点都位于间隔线 $y_i(w^{*\top} x_i + b^*) = 1$ 上，将这些点称为 **support vector**（支撑向量）。去掉数据集除支撑向量外的任何点都不会影响分离超平面。

对于间隔线，具体来说，若 $y_i = 1$ ，则有 $w^\top x_i + b = 1$ ；若 $y_i = -1$ ，则有 $w^\top x_i + b = -1$ 。在间隔线两侧的点就是正确区分的点，间隔线中间就是分离超平面 $w^\top x_i + b = 0$ ，整个间隔宽度为 $\frac{2}{\|w\|}$ （因为

$$w^\top x = w^\top (x^{(1)} + x^{(2)}) = -b + w^\top x^{(2)} \Rightarrow \|x^{(2)}\| = \frac{|w^\top x + b|}{\|w\|}。$$

将站点条件和互补松弛条件代入拉格朗日函数，可以得到：

$$L(w, b, \alpha) = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) + \sum_{i=1}^n \alpha_i$$

从而得到容易处理的对偶问题：

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^n \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha_i \geq 0 \end{aligned}$$

3.4 软间隔最大化

事实上很少有可以完全线性可分的数据，因此我们需要给优化目标添加一定的“犯错容忍度” $\xi_i \geq 0$ ：

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, n \end{aligned}$$

同一套流程走一遍，得到结果：

$$\begin{cases} \nabla_w L = 0 \Rightarrow w^* = \sum_i \alpha_i^* y_i x_i & \text{站点条件} \\ \nabla_b L = 0 \Rightarrow \sum_i \alpha_i^* y_i = 0 \\ \nabla_{\xi_i} L = 0 \Rightarrow C - \alpha_i^* - \mu_i^* = 0 \\ \alpha_i^* [1 - \xi_i^* - y_i(w^{*\top} x_i + b^*)] = 0, \mu_i^* \xi_i^* = 0 & \text{互补松弛} \\ y_i(w^{*\top} x_i + b^*) \geq 1 - \xi_i^*, \xi_i^* \geq 0 & \text{原始约束} \\ \alpha_i^* \geq 0, \mu_i^* \geq 0 & \text{对偶可行} \end{cases}$$

容易知道 $0 \leq \alpha_i \leq C$ ，故分三种情况讨论：

1. 若 $\alpha_i^* = 0$ ，则有 $\mu_i^* = C \Rightarrow \xi_i^* = 0$ ，故 $y_i(w^{*\top} x_i + b) \geq 1$ 。意味着点在支撑超平面或支撑超平面以外。
2. 若 $\alpha_i^* < C$ ，则有 $\mu_i^* = C - \alpha_i^* > 0 \Rightarrow \xi_i^* = 0$ ，故 $y_i(w^{*\top} x_i + b^*) = 1$ 。意味着点在支撑超平面上。
3. 若 $\alpha_i^* = C$ ，则有 $\mu_i^* = 0 \Rightarrow \xi_i^* \geq 0$ ，故 $y_i(w^{*\top} x_i + b^*) = 1 - \xi_i^*$ 。进一步细分，若 $\xi_i^* = 0$ ，说明点在支撑超平面上；若 $0 < \xi_i^* < 1$ ，说明点在支撑超平面和分离面之间；若 $\xi_i^* = 1$ ，说明点在分离面上；若 $\xi_i^* > 1$ ，说明点在分离面另一侧，意味着属于错分类。

在支撑超平面之间的点到对应间隔边界的距离为 $\frac{\xi_i}{\|w\|}$ 。

3.5 合页损失函数

之前我们采用 KKT 条件来处理对偶问题，最后得到关于 α 的优化问题。但其实有另一种的处理方法：

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, n \end{aligned}$$

可以得到

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i = \min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n [1 - y_i(w^\top x_i + b)]_+$$

等价于最小化

$$L(x, y) = [1 - y(w \cdot x + b)]_+$$

同时添加正则项 $\frac{1}{2} \|w\|^2$.

3.6 非线性 SVM 与核函数

对于非线性可分问题，通常将点积换为核函数：

$$\begin{aligned} W(\alpha) &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^n \alpha_i \\ \rightarrow W(\alpha) &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \kappa(x_i, x_j) - \sum_{i=1}^n \alpha_i \end{aligned}$$

正定核的充要条件： $\mathcal{K} = [\kappa(x_i, x_j)]_{i=1, j=1}^{n, n}$ 半正定

常用的核函数：

- $\kappa(x, z) = (x \cdot z + 1)^p$
- $\kappa(x, z) = \exp(-\frac{\|x - z\|^2}{2\sigma^2})$

5. 手动实现自动微分

5.1 梯度、导数

梯度：

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{bmatrix}$$

其中

$$\frac{\partial f}{\partial x_j}(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{e}_j) - f(\mathbf{x})}{h}$$

导数：

$$D_{\mathbf{v}} f(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{v}) - f(\mathbf{x})}{h}$$

联系：

$$D_{\mathbf{v}} f(\mathbf{x}) = \nabla f(\mathbf{x}) \cdot \mathbf{v}$$

Proof:

Let $g(t) = f(\mathbf{x} + t\mathbf{v})$ ，有 $g'(t) = \nabla f(\mathbf{x} + t\mathbf{v}) \cdot \mathbf{v}$ ，又因为 $g'(0) = D_{\mathbf{v}} f(\mathbf{x})$ ，得证。

5.2 VJP & JVP

Vector-Jacobian Product(VJP) 和 Jacobian-Vector Product(JVP) 分别对应反向模式和正向模式的自动微分。

设函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ，其雅可比矩阵 $J_f = (\nabla f)^\top$ 是一个 $m \times n$ 矩阵。显式存储成本为 $O(mn)$ ，当 n 或 m 很大时，直接计算不可行。

$$\nabla f(\mathbf{x}) = J_g(\mathbf{x})^\top \nabla h(g(\mathbf{x}))$$

VJP: $v^\top J_f \in \mathbb{R}^{1 \times n}$

- $\mathbf{v}^\top J_{f_4}(\mathbf{x}_4) J_{f_3}(\mathbf{x}_3) J_{f_2}(\mathbf{x}_2) J_{f_1}(\mathbf{x}_1)$
- 前向计算后通过链式法则反向传播
- 当输出维度 m 较小时（如标量损失函数）高效，复杂度 $O(m)$
- 适用场景：深度学习

JVP: $J_f u \in \mathbb{R}^m$

- $J_f(\mathbf{x})\mathbf{u} = J_{f_4}(\mathbf{x}_4) J_{f_3}(\mathbf{x}_3) J_{f_2}(\mathbf{x}_2) J_{f_1}(\mathbf{x}_1)\mathbf{u}$
- 前向计算时同时传入输入方向的向量 u
- 当输入维度 n 较小时高效，复杂度 $O(n)$
- 适用场景：物理模拟

6. MLP

MLP:

$$o = Wx + b$$

梯度：

$$\begin{aligned} \frac{\partial l}{\partial W_{ij}} &= x_j \frac{\partial l}{\partial o_i} \Rightarrow \frac{\partial l}{\partial W} = \frac{\partial l}{\partial o} \cdot x^\top \\ \Rightarrow \frac{\partial L}{\partial W} &= \frac{1}{N} \sum_{i=1}^N \frac{\partial l_i}{\partial W} = \left(\frac{\partial L}{\partial O} \right)^\top X \\ \frac{\partial L}{\partial b} &= \left(\frac{\partial L}{\partial O} \right)^\top \mathbf{1}_N \end{aligned}$$

Xavier Initialization:

$$\gamma_t \cdot \frac{n_{t-1} + n_t}{2} = 1 \Rightarrow \gamma_t = \frac{2}{n_{t-1} + n_t}$$

Normal distribution: $\mathcal{N}(0, \sqrt{\frac{2}{n_{t-1} + n_t}})$

Unifrom distribution: $\mathcal{U}(-\sqrt{\frac{6}{n_{t-1} + n_t}}, \sqrt{\frac{6}{n_{t-1} + n_t}})$

7. 过拟合防止方法

7.1 Early Stop

训练过程中，在测试集精度停止提高(-> trigger 触发)后终止训练。三个要点：

- Monitor model performance
- Trigger to stop training
- The choice of model to use

可以将训练集中随机分出 30% 的数据作为验证集用来监控训练中的模型性能。

可采用的 triggers：

- No change in metric over a given number of epochs
- An absolute change in a metric
- A decrease in performance observed over a given number of epochs
- Average change metric over a given number of epochs

根据不同的 trigger 采用不同的保存策略。

7.2 Normalization

假设为 ReLU 网络初始化 $W_i \sim \mathcal{N}(0, \frac{c}{n})$ ，结果： $c = 3 \rightarrow \text{NaN}$, $c = 2 \rightarrow \text{Works}$, $c = 1 \rightarrow \text{No progress}$

Idea 1: Layer Normalization 层归一化

$$\hat{z}_{i+1} = \sigma_i(W_i^\top z_i + b_i)$$
$$z_{i+1} = \frac{\hat{z}_{i+1} - \mathbb{E}[\hat{z}_{i+1}]}{(\text{Var}[\hat{z}_{i+1}] + \epsilon)^{\frac{1}{2}}}$$

解决内部协变量偏移问题(Internal Covariate Shift)。

Internal Covariate Shift: 在深度神经网络训练过程中，由于网络中间层的参数更新，导致每一层的输入分布不断变化。这种变化会影响后续层的训练，使得每一层需要不断适应这些变化，从而导致训练过程变得更加缓慢、不稳定，甚至影响最终的模型性能。

Idea 2: Batch Normalization 批量归一化

也能解决上述问题。并且实践中通常比 layer norm 效果更好。

带来的问题：Minibatch dependence（小批量依赖）

解决方法：在训练时，使用每个小批量的均值和方差进行归一化；而在测试时（或者在推理阶段），不再使用当前批次的均值和方差，而是使用训练阶段计算得到的全局均值和方差。

可以有效降低预测对 batch 的依赖性，从而减少噪声，提高稳定性。

7.3 Interaction of Optimization, Initialization, Normalization

BN 层的作用存在争议。好的初始化即使没有 BN 层，效果也比有 BN 层的随机初始化强（50 layers FCN）。

7.4 Regularization

限制模型复杂度。两种方法：

- Implicit regularization（隐式正则化）：在算法中实现
- Explicit regularization（显示正则化）：对网络和训练过程进行修改

隐式正则化本质上是正则化项求导后的 weight decay 项。

$$W_i \leftarrow (1 - \alpha\lambda)W_i - \alpha \nabla_{W_i} \ell(h(X), y)$$

7.5 Dropout

$$\begin{aligned} \hat{z}_{i+1} &= \sigma_i(W_i^\top z_i + b_i) \\ (z_{i+1})_j &= \begin{cases} \frac{(\hat{z}_{i+1})_j}{1-p} & \text{with probability } 1-p \\ 0 & \text{with probability } p \end{cases} \\ \Rightarrow z_{i+1} &= \sigma \left(\frac{n}{|\mathcal{P}|} \sum_{j \in \mathcal{P}} W_{:,j} (z_i)_j \right) \end{aligned}$$

能让网络稳健性(robust)提升，本质上是一种随机近似(stochastic approximation)。

注意：测试时需要停用 Dropout 层。当然有的研究者在测试时也使用 Dropout 层，来启发式判断模型预测的不确定性。

在 CNN 上可以使用 Spatial Dropout，丢弃整个特征通道；在 RNN 上可以使用 Variational Dropout，在每个 step 均使用同样的 dropout mask 以保持时间一致性。

变种：

- Alpha Dropout: used with the SELU activation function
- Concrete Dropout: a self-adaptive dropout technique
- Monte Carlo Dropout: used in Bayesian neural networks

7.6 Data Augmentation

提高数据利用率，提高数据多样性。

使用情景：

- 防止过拟合
- 初始数据集过小
- 提高模型精度
- 减少数据集标注和数据集清洗的开销

限制：

- 增强数据中的偏差
- 数据增强的质量检测是昂贵的
- 不成熟，缺乏相关研究和技术开发
- 寻找高效的数据增强方法是具有挑战性的

Audio Data Augmentation:

- Noise injection
- Shifting
- Changing the speed
- Changing the pitch

Text Data Augmentation:

- Word or sentence shuffling
- Word replacement
- Syntax-tree manipulation
- Random word insertion
- Random word deletion

Image Augmentation:

- Geometric transformations
- Color space transformations
- Kernel filters
- Random erasing
- Mixing images

Advanced Techniques:

- Generative adversarial networks (GANs)
- Neural Style Transfer

8. 构建神经网络库

8.1 Programming Abstractions

前向和反向层接口 (Forward and backward layer interface) :

```
class Layer:
    def forward(bottom, top):
        pass

    def backward(top, propagate_down, bottom):
        pass
```

计算图和声明式编程 (Computational graph and declarative programming) → **静态** e.g. TensorFlow

命令式自动微分 (Imperative automatic differentiation) → **动态** e.g. Pytorch

8.2 High level modular library components

机器学习三大要素:

- The hypothesis class
- The loss function
- An optimization method

More details:

- Model and architecture
- Objective function and training techniques e.g. Supervised, self-supervised, RL, adversarial learning (对抗学习)
- Regularization, normalization, initialization e.g. Batch norm, dropout, Xavier
- Get good amount of data

深度学习是 模块化的 (modular), 其中 **损失函数** 是作为一种特殊的模块。

Optimizers:

- SGD:

$$w_i \leftarrow w_i - \alpha g_i$$

- SGD with momentum:

$$u_i \leftarrow \beta u_i + (1 - \beta) g_i$$
$$w_i \leftarrow w_i - \alpha u_i$$

- Adam:

$$\begin{aligned}
 u_i &\leftarrow \beta_1 u_i + (1 - \beta_1) g_i \\
 v_i &\leftarrow \beta_2 v_i + (1 - \beta_2) g_i^2 \\
 w_i &\leftarrow w_i - \frac{\alpha u_i}{\sqrt{v_i} + \varepsilon}
 \end{aligned}$$

Regularization:

- Implement as part of loss function
- Directly incorporate as part of optimizer update

第二种方式实际上相当于带权重衰减(weight decay)项的优化器。(见 7.4)

Initialization:

取决于模型和参数。一般将偏置项置零，权重的数量级一般取决于输入/输出。

Data loader and preprocessing:

通常结合在一起作为一个模块。